

f-Strings in Python

f-strings (formatted string literals) are an easy and modern way to format strings in Python. They were introduced in **Python 3.6**.

To use an f-string, you add an f before the string and place variables or expressions inside {}.

Basic Example

```
name = "Harry"
```

```
age = 25
```

```
print(f"My name is {name} and I am {age} years old")
```

👉 Here, {name} and {age} are automatically replaced with their actual values.

Why f-Strings?

- Cleaner than string concatenation (+) and .format()
- Faster and easier to read
- Supports expressions and method calls

Strings & String Methods in Python

Strings are used to store text data in Python.

Any value written inside single quotes or double quotes is treated as a string.

Strings are one of the most commonly used data types and Python provides many built in methods to work with them.

Creating Strings

Strings can be created using single quotes or double quotes.

```
name = "Harry"
```

```
city = 'Delhi'
```

Both forms work the same way.

Multiline Strings

Multiline strings are created using triple quotes.

```
message = """This is a
```

```
multi line
```

```
string"""
```

Accessing Characters in a String

Strings are indexed, which means each character has a position.

```
text = "Python"
```

```
print(text[0])
```

```
print(text[3])
```

Indexing starts from 0.

String Length

You can find the length of a string using the len() function.

```
text = "Python"
```

```
length = len(text)
```

String Slicing

Slicing is used to extract a portion of a string.

```
text = "Python"
print(text[0:3])
print(text[2:])
print(text[:4])
```

When slicing a string with negative indexes, Python internally adds the length of the string to the negative value.

```
text[-3:] # same as text[6-3 : 6]
text[-5:-2] # same as text[6-5 : 6-2]
```

So Python converts negative indexes like this:

```
-3 -> 6 - 3 = 3
-5 -> 6 - 5 = 1
-2 -> 6 - 2 = 4
```

Common String Methods

Python provides many built in string methods for common operations.

`lower()` **and** `upper()`

Convert string to lowercase or uppercase.

```
text = "Python"
print(text.lower())
print(text.upper())
```

`strip()`

Removes extra spaces from the beginning and end of a string.

```
text = " hello "
print(text.strip())
```

`replace()`

Replaces part of a string with another string.

```
text = "Hello World"
```

```
print(text.replace("World", "Python"))
```

`split()`

Splits a string into a list based on a separator.

```
text = "apple,banana,orange"
```

```
items = text.split(",")
```

`join()`

Joins elements of a list into a single string.

```
items = ["apple", "banana", "orange"]
```

```
text = ";".join(items)
```

`find()`

Finds the position of a substring.

```
text = "Hello World"
```

```
position = text.find("World")
```

Returns -1 if the value is not found.

`startswith()` **and** `endswith()`

Checks whether a string starts or ends with a given value.

```
email = "test@gmail.com"
```

```
print(email.startswith("test"))
```

```
print(email.endswith(".com"))
```

String Concatenation

Strings can be combined using the + operator.

```
first = "Hello"
```

```
second = "World"
```

```
result = first + " " + second
```

String Formatting

String formatting allows inserting values into strings.

Using f strings

```
name = "Harry"
```

```
age = 25
```

```
message = f"My name is {name} and I am {age} years old"
```

Checking String Content

Python provides methods to check string content.

```
text = "Python123"
```

```
print(text.isalpha())
```

```
print(text.isdigit())
```

```
print(text.isalnum())
```

Strings are Immutable

Strings cannot be changed after creation.

```
text = "Python"
```

```
text[0] = "J" This will cause an error. Any modification creates a new string.
```